B-Bäume

Ausgeglichene Mehrweg-Suchbäume

Motivation & Grundidee

Speicher	Zugriffszeit	Größe
Hauptspeicher	10 – 50 ns	10^{-8} s
Flash / SSD	10 – 250 µs	10^{-4} s
Festplatte (HDD)	3 – 20 ms	10^{-3} s

Ziel

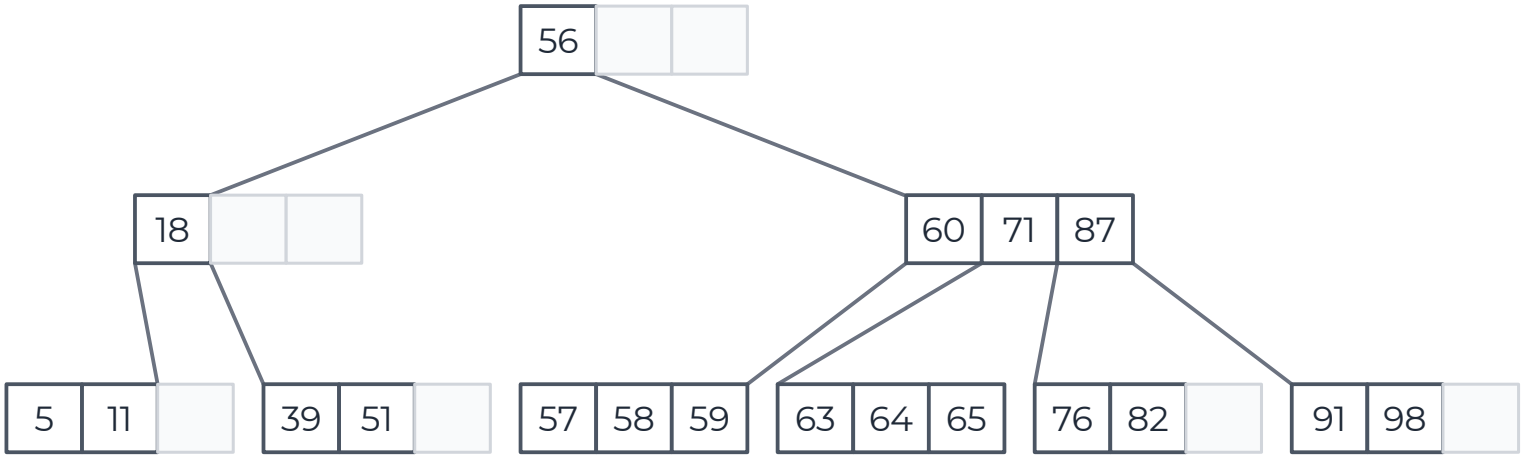
möglichst **wenige Zugriffe** auf den externen Speicher

- **Vollständig ausgeglichen**: alle Blätter liegen auf derselben Ebene
- Viele Elemente pro Knoten → **geringe Höhe** → wenige Zugriffe
- Ein Knoten = eine **Seite** (Block) des Speichermediums
- Einsatz: **Indizes** in relationalen Datenbanken

B-Baum der Ordnung d

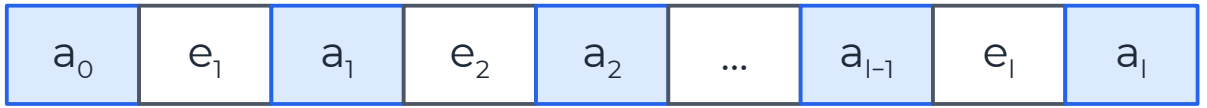
	Nachfolger	Elemente
Wurzel	2 bis d	1 bis d-1
Blätter	0	$\lfloor \frac{d-1}{2} \rfloor$ bis d-1
sonstige Knoten	$\lceil \frac{d}{2} \rceil$ bis d	$\lfloor \frac{d-1}{2} \rfloor$ bis d-1

Beispiel: Ordnung 4



- Jede Seite hat höchstens **4** Nachfolger und höchstens 4 − 1 = **3** Elemente
- Jede Seite (≠ Blatt) hat genau **einen Nachfolger mehr** als Elemente
- Jede Seite (≠ Wurzel) hat mindestens ⌊(4−1)/2⌋ = **1** Element

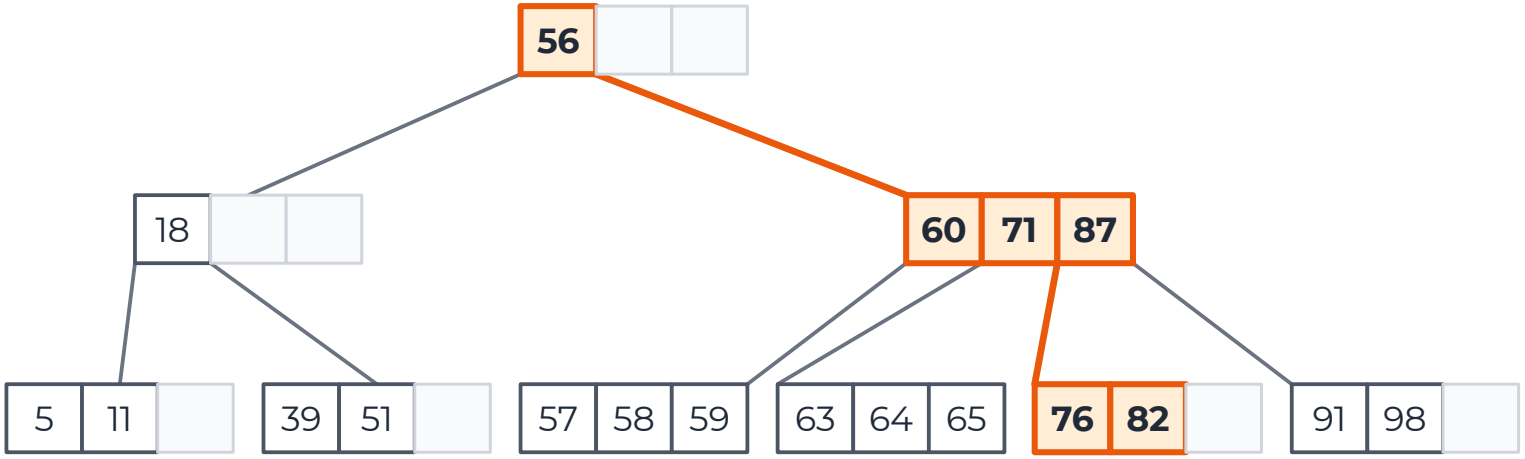
Aufbau einer Seite



Zeiger (Seitenadressen)
Elemente (sortiert)

- Eine Seite enthält l Elemente $e_1 < e_2 < ... < e_l$ und l + 1 Zeiger a_0 ... a_l
- Jeder Zeiger a_i verweist auf die Wurzel eines **Teil-B-Baums**
- Wertebereiche der Teilbäume:
  - bei a_0: alle Elemente **<** e_1
  - bei a_i: alle Elemente **zwischen** e_i und e_(i+1)
  - bei a_l: alle Elemente **>** e_l
- Es gilt $\lfloor (d-1)/2 \rfloor \leq l \leq d - 1$ (Wurzel: $l \geq 1$)

Suchen: Beispiel 76



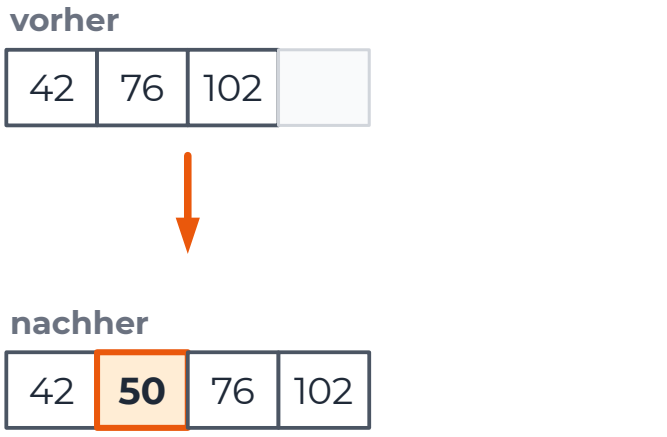
1. Wurzel [56]: 76 > 56 → rechter Zeiger
2. [60 | 71 | 87]: 71 < 76 < 87 → Zeiger zwischen 71 und 87
3. [76 | 82]: **76 gefunden** ✓

Wird e **nicht** gefunden, endet die Suche immer in einem **Blatt**.

Einfügen: Ablauf

1. Mit der Suche das **Blatt** finden, in das e gehört
2. Blatt **nicht voll**: e sortiert einfügen – fertig
3. Blatt **voll** (Overflow): Seite **teilen** (Splitting). Das mittlere Element wandert in die Vorgängerseite
4. Vorgängerseite jetzt auch übervoll? → wieder Splitting, bis zur Wurzel
5. Wurzel geteilt: **neue Wurzel** → der Baum wird eine Ebene höher

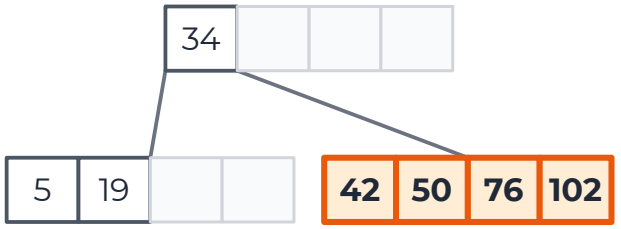
Beispiel d = 5: Einfügen von 50



Blatt hatte noch Platz (max. 4 Elemente)

Einfügen: Overflow & Splitting

1. Einfügen von 83: Blatt ist voll

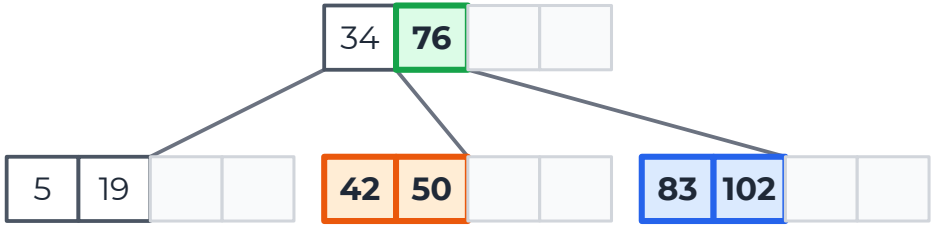


2. Übergelaufene Seite (nur gedanklich)



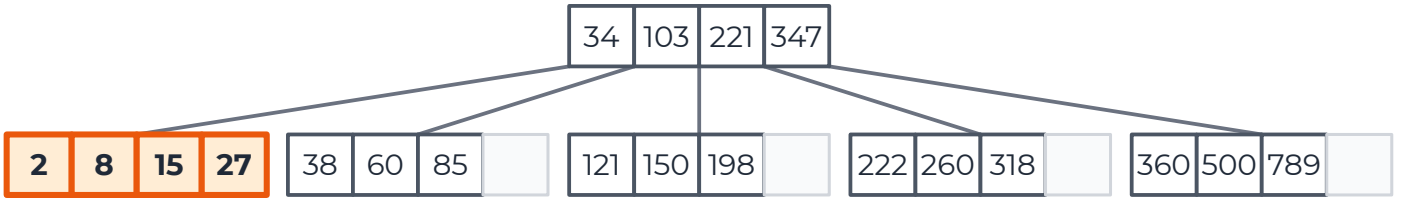
- Mittleres Element ⌈d/2⌉ = 3 → **76** wandert nach oben
- **42, 50** bleiben in der alten Seite
- **83, 102** kommen in eine neue Seite
- 76 wird in die Vorgängerseite einsortiert, mit Zeiger auf die neue Seite

3. Nach dem Splitting



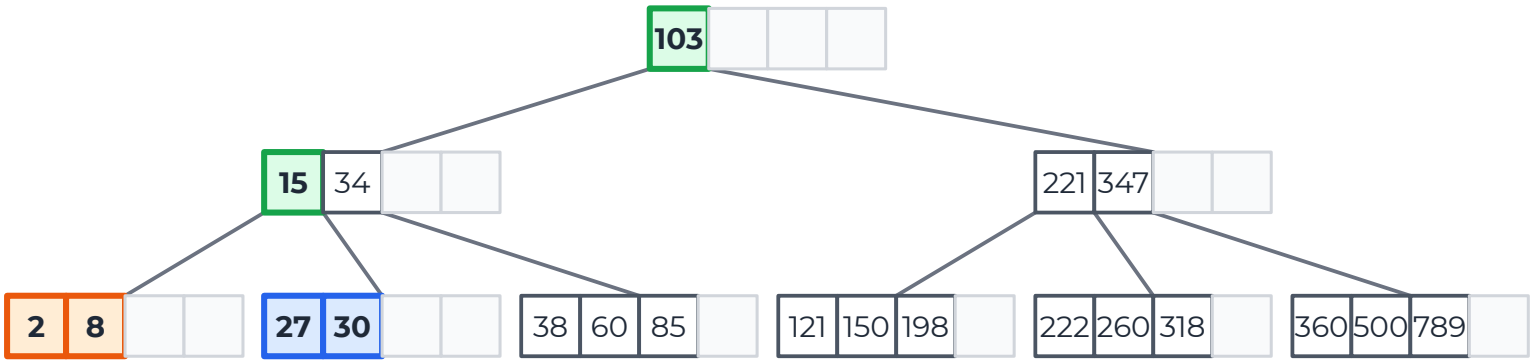
Einfügen: Wurzel teilen

Vorher: Einfügen von 30 (Blatt [2, 8, 15, 27] ist voll, Wurzel ist voll)



1. Blatt teilen: **15** steigt auf
2. Wurzel teilen: **103** steigt auf

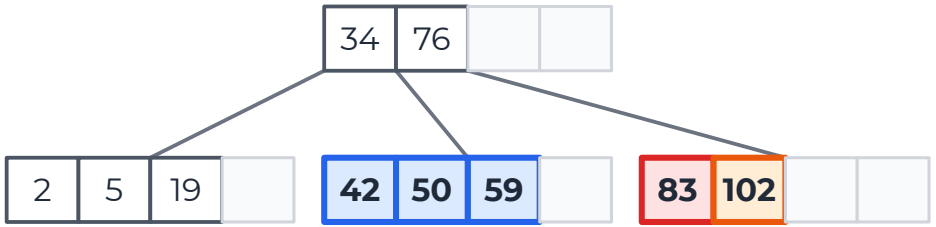
Nachher: zwei Splits, die Wurzel ist neu



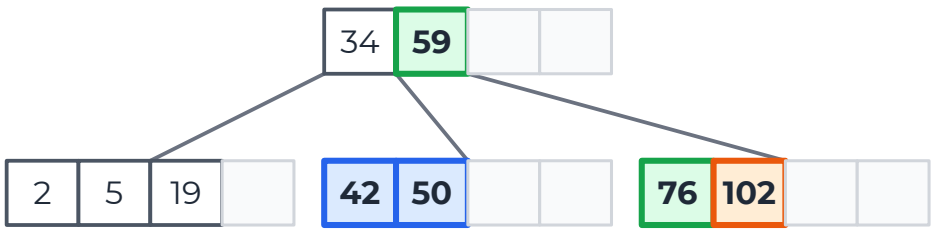
Der Baum wächst an der **Wurzel** – deshalb bleiben alle Blätter auf einer Ebene.

Löschen: Ausgleichen (83)

Vorher: lösche 83



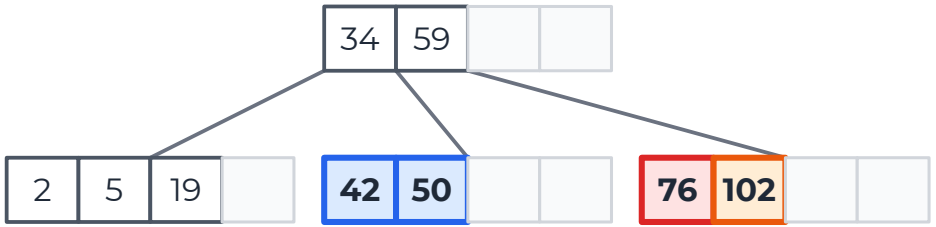
Nachher: 59 wandert nach oben, 76 nach unten



- 83 gelöscht → Seite [102] hat nur noch **1** Element
- Minimum bei d = 5: **2** Elemente → **Underflow**
- Linke Nachbarseite [42, 50, 59] hat **3** Elemente (> Minimum)
- **Ausgleichen:** 59 steigt in die Elternseite, 76 sinkt in die Seite mit 102

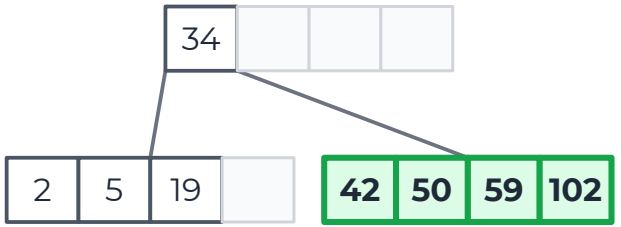
Löschen: Verschmelzen (76)

Vorher: lösche 76



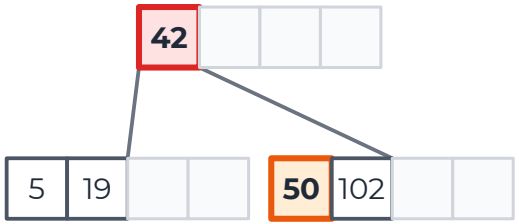
- 76 gelöscht → Seite [102] hat nur noch **1** Element → **Underflow**
- Linke Nachbarseite [42, 50] hat **genau** das Minimum → kann nichts abgeben
- **Verschmelzen:** [42, 50] + 59 + [102] = [42, 50, 59, 102]
- Die Elternseite verliert ein Element (59)

Nachher: Seiten verschmolzen, Elternseite verliert 59

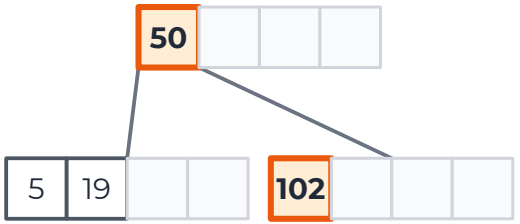


Löschen: Baum schrumpft (42)

1. lösche 42 → Nachfolger 50



2. Underflow im Blatt



3. Verschmelzen: neue Wurzel



- 42 steht in der Wurzel → durch den **symmetrischen Nachfolger** 50 ersetzen (gelöscht wird im Blatt)
- Blatt [102] hat einen **Underflow**; Nachbar [5, 19] hat genau das Minimum → **Verschmelzen**: [5, 19] + 50 + [102]
- Die Wurzel ist jetzt leer → die verschmolzene Seite wird die **neue Wurzel**: der Baum ist um eine Ebene **geschrumpft**